

FORGE / TECHNICAL EXTENSION STUDY

Exact Infinite-State Workers for Forge

Antichains, equality orbits,
and their checked product

A proposal with executable searches, independent replay paths,
and explicit Lean implementation obligations

Prepared for Vladimir Reshetnikov
15 September 2026

Repository baseline. Forge 58ea206bd7ad; Leant 6bf05ad78c46; Djex e8778f4ebd63. Full revision identifiers and inspected paths appear in the source ledger.

Executed here. Three standard-library Python workers; 27 unit-test methods; 640 randomized differential cases; 650 stored certificate objects; 1,300 rejected structural or required-content mutations.

Not established here. A Lean tactic, a verified Lean checker, a compiled Lean source bridge, or any solved-goal or speed advantage over another tactic. The included small Lean arithmetic specimen is explicitly uncompiled.

Abstract

Forge’s inspected design already has substantial machinery for induction, witness synthesis, algebraic and analytic certificates, finite-dimensional closures, and pushdown summaries. Repeating that architecture would add little. This article instead specifies two semantic workers and their exact composition. The first computes backward-closed unsafe regions of finite-control counter systems using minimal antichains, deciding coverability without bounding the counters and admitting whole nonnegative affine families of initial states. The second decides equivalence of deterministic equality-register transducers over an unbounded alphabet by exploring joint equality orbits, not concrete keys. Their product handles systems with both a bounded set of remembered names and an unbounded vector of resource counts.

Each worker has a finite positive receipt, an executable negative witness, a separate checker, explicit source-translation requirements, and a precise boundary beyond which it must return unknown or unsupported. Constructive proofs explain the predecessor formula, parameter separation, fresh-input coverage, finite-alphabet cutoff, and mixed-model transport. A two-key exhaustive cache test is shown to miss a three-key, five-request counterexample. The delivered experiments also include parameterized mutual exclusion, nonconvex unbounded safety, and an ownership protocol with arbitrarily many waiting clients. The algorithms have classical foundations; the proposed contribution is their concrete, narrowly typed integration into Forge, together with reproducible implementation evidence rather than an assertion of a new model-checking theory.

Contents

1	The extension, not another architecture	3
1.1	What the repository already supplies	3
1.2	What makes the new workers useful	3
1.3	Historical versus project novelty	4
1.4	Three deliberately narrow interfaces	4
2	Backward antichains for unbounded counters	4
2.1	Exact fragment and semantics	4
2.2	The exact predecessor formula	5
2.3	Parameter separation without a solver	6
2.4	Search by minimal unsafe configurations	6
2.5	A positive certificate smaller than the search	7
2.6	Why witness history must outlive subsumption	8
2.7	Worked unbounded examples	8
3	Equality orbits instead of a finite test alphabet	9
3.1	The distinction that finite tests miss	9
3.2	A small, typed machine language	9
3.3	Joint support and canonical representatives	9
3.4	Why one fresh representative is enough	10
3.5	Search and finite certificates	11
3.6	Termination, cutoff, and witness length	11
3.7	Worked example: the cache distinction	12
3.8	Checking cost and representation choices	13
4	Names and counters in one exact worker	13
4.1	The admitted mixed language	13
4.2	Compile structural name orbits first	13
4.3	Check against the mixed source, not the compiler output	14

4.4	Worked example: a remembered owner and arbitrary clients	15
4.5	A finite oracle for differential testing, not for proving this case	15
5	Lean integration at the actual Forge boundary	15
5.1	Reuse the existing contracts	15
5.2	A proof-carrying frontend for counter programs	16
5.3	A proof-carrying frontend for equality programs	17
5.4	Registering summaries of reusable primitives	17
5.5	Candidate construction and proof replay	18
5.6	How Leant and Djex fit, without overclaiming their problems	18
6	Executed evidence and its limits	19
6.1	What was run	19
6.2	What the differential oracles actually do	19
6.3	Workload size and construction bias	20
6.4	Handcrafted results	20
6.5	Mutations and positive robustness controls	21
6.6	Replay separation and remaining risks	21
7	Concrete libraries, engineering plan, and acceptance gates	21
7.1	Library choices and what not to outsource	21
7.2	Producer data structures	23
7.3	A phased implementation with falsifiable gates	23
7.4	Routing and useful refusal diagnostics	24
7.5	A restrained direction beyond this delivery	24
A	Reproducing and using the artifacts	24
B	Model and certificate schema notes	25
C	Source audit and implementation status	26
D	Concluding assessment	27
	References	27

1 The extension, not another architecture

1.1 What the repository already supplies

The baseline for this article is the inspected Forge revision 58ea206bd7ad (full identifier in Appendix C), not the initial proposal to improve `grind`. Its README and provenance ledger describe a merge of 27 proposals across three rounds. The merged design already includes structural proof planning, generalization, synthesized witnesses and invariants, observable-space and ideal closure, finite-algebra covers, tree summaries and observable quotients, summation and Ore transport, noncommutative algebra, analytic enclosures, quantitative probabilistic workers, games, and unbounded pushdown summaries [1]. A generic instruction to add induction, a CAS, an e-graph, or a certificate boundary would therefore be a repetition.

Nor should a new report claim that none of Forge’s Lean has ever compiled. The README records 18 core-only files elaborated by the merge, even though the contributing proposal environments did not run Lean. Those elaborations do not establish an implemented `forge` tactic or a verified family of certificate checkers. The present environment likewise has no Lean executable; that is a limitation of this delivery, not a revision of the repository’s own evidence.

The audit used the merged article’s closure section and provenance inventory, the README, and the concrete runtime, logical-contract, and finite-cover Lean files. The Leant/Djex comparison used their current revision information and indexing-composition diagnostics. It was not a line-by-line review of every archived proposal. In particular, the empty results returned by the repository code-search endpoint were not treated as evidence of absence: even a known baseline term returned no match. The delta claims below are relative to the inspected merged design, with the nearest existing capability named explicitly.

Addition	Nearest existing Forge mechanism	Actual additional work
Counter antichains	Numeric invariants, ideal closure, and finite covers	Compute exact backward coverability in $Q \times \mathbb{N}^d$; retain consuming guards; certify all members of nonnegative affine initial families.
Equality orbits	Finite-algebra covers and observable quotients	Construct the finite quotient for equality-only data over infinitely many atoms; prove joint support and fresh-case coverage; recover concrete witnesses.
Orbit-counter product	Separate infinite-state workers	Compile finite name orbits to controls while retaining unbounded counters, and check directly against the original mixed-rule semantics.
Candidate discharge	Existing typed synthesis and behavioral filtering	Use these exact semantic summaries after candidate reconstruction, with a proved source bridge, rather than promote finite observations to a theorem.

1.2 What makes the new workers useful

A finite set of *concrete states* cannot cover an arbitrarily large pool of waiting clients. A finite set of *minimal bad configurations* can describe all markings from which a violation is possible.

Similarly, finitely many concrete keys do not represent every name a program might receive, but finitely many equality patterns relative to its live registers do.

These are different reasons for finiteness. For counters, finiteness comes from a well quasi-order on natural vectors. For names, it comes from quotienting by renamings that preserve everything the program can observe. Combining them requires a precise intermediate language; it is not justified merely by saying that both components separately have decidable fragments.

The intended Forge results are ordinary mathematical theorems, for example

$$\forall n \in \mathbb{N}, \forall \text{ finite executions from } s_0(n), \quad \text{active} \leq 1,$$

or

$$\forall w \in (\text{Tag} \times \mathbb{A})^*, \quad \text{outputs}_A(w) = \text{outputs}_B(w).$$

The quantifiers range over all initial resource sizes and all finite input lengths. Neither is replaced by a test bound. The finite object returned by a worker explains why those quantifiers can be discharged.

1.3 Historical versus project novelty

The mathematical foundations are established: monotone systems and well-quasi-order methods for coverability [6, 7], and automata over nominal sets for data alphabets with symmetries [8]. We do not rename these results and claim to have invented them. The contribution here is a specific fragment choice, exact certificate formats, an initial-family interface, an independently computed replay path, a mixed-rule frontend, and an integration plan tied to Forge’s actual types.

This distinction also prevents an exaggerated expressiveness claim. A hand-made induction invariant may prove the mutual-exclusion example, and the repository’s polynomial machinery could express some of the same properties. A complete specialized worker is still useful when it discovers the finite invariant without choosing a template, emits a counterexample when the property is false, and explains exactly when its answer is exhaustive. No delivered experiment compares it with existing Lean tactics.

1.4 Three deliberately narrow interfaces

The executable workers accept mathematical intermediate representations, not arbitrary Python or Lean programs. Their admission conditions are part of their meaning. A counter system has nonnegative integer consumption and production, upward-closed bad sets, and no hidden tests. An equality machine has only finite control, fixed constants, equality tests, and copying registers. A mixed system has both components, but counter updates do not depend on the numerical identity of an atom and do not allocate an unbounded map of counters indexed by names.

The source program remains outside this small language until a proved bridge connects its execution to the representation. That bridge is where most Lean implementation effort belongs. Another search heuristic cannot make an unjustified translation sound.

2 Backward antichains for unbounded counters

2.1 Exact fragment and semantics

Fix a finite set of control locations Q and $d \geq 1$ counters. A state is (q, x) with $x \in \mathbb{N}^d$. A transition is a tuple

$$e = (q, a, z, q'), \quad a, z \in \mathbb{N}^d,$$

with semantics

$$(q, x) \xrightarrow{e} (q', x - a + z) \iff x \geq a. \quad (1)$$

All vector comparisons are componentwise. Consumption happens before production. This is a finite-control place/transition-net representation; it can encode a vector addition system by choosing appropriate input and output arcs, but input and output arcs must not be collapsed without preserving the enabling test.

A finite collection $B_q \subseteq \mathbb{N}^d$ specifies the unsafe target at location q :

$$\text{Bad}_q = \uparrow B_q := \{x \mid \exists b \in B_q, b \leq x\}.$$

Thus the question is whether a target marking can be *covered*, not whether a prescribed vector can be reached with exact equality in every coordinate. For example, “at least one process is in each of two critical sections” is upward-closed. A condition such as “counter i equals zero” generally is not.

Initial states may form a finite union of families

$$X_f = \left\{ b_f + \sum_{j=1}^{m_f} n_j r_{f,j} : n_j \in \mathbb{N} \right\}, \quad r_{f,j} \in \mathbb{N}^d, \quad (2)$$

with a supplied control location q_f . Zero rays and $m_f = 0$ are allowed. All parameters are independent. Correlated or bounded parameters, negative coefficients, and side constraints are different interfaces, not implicit extensions of this one.

2.2 The exact predecessor formula

For one target threshold b , the set of predecessors under e is itself an upward cone. Define

$$\text{pred}_e(b) = a + (b - z)^+ = \max(a, a + b - z), \quad (3)$$

where positive part and maximum are componentwise. The expression $a + b - z$ on the right is interpreted over integers; $b - z$ on the left may equivalently be natural-number truncated subtraction.

Lemma 2.1 (Enabling-aware predecessor). *For $x, a, z, b \in \mathbb{N}^d$,*

$$x \geq a \wedge x - a + z \geq b \iff x \geq \text{pred}_e(b).$$

Consequently $\text{Pre}_e(\uparrow b) = \uparrow \text{pred}_e(b)$.

Proof. Work in one coordinate. Under $x \geq a$, the second inequality is equivalent to $x \geq a + b - z$ over integers. Together the two inequalities say $x \geq \max(a, a + b - z) = a + \max(0, b - z)$. The argument applies to every coordinate. $\square$

Example 2.2 (The delta-only bug). There are two controls and one transition from control 0 to control 1, consuming one token and producing one token. Initially the state is $(0, 0)$. Every state at control 1 is bad. The net change is zero, but the transition is disabled. The correct backward basis is $C_1 = \{0\}$ and $C_0 = \{1\}$, so the initial state is safe. A formula using only the net change would give predecessor threshold zero and invent a nonexistent counterexample. The fixture `enabling` checks exactly this case.

2.3 Parameter separation without a solver

A receipt must be disjoint from *all* states in an initial family, not just its base. This check has a simple exact form because the rays are nonnegative.

Lemma 2.3 (Affine-family separation). *Let $X = b + \sum_j \mathbb{N}r_j \subseteq \mathbb{N}^d$, and let $c \in \mathbb{N}^d$. Then $X \cap \uparrow c = \emptyset$ if and only if there is a coordinate i such that*

$$b_i < c_i \quad \text{and} \quad (r_j)_i = 0 \text{ for every } j.$$

Proof. Such a coordinate stays below c_i for every parameter assignment, proving separation. Conversely suppose no such coordinate exists. Write $s_i = \sum_j (r_j)_i$. Whenever $b_i < c_i$, we have $s_i > 0$. Choose

$$t = \max_{i: b_i < c_i} \left\lceil \frac{c_i - b_i}{s_i} \right\rceil, \quad \max \emptyset = 0, \quad (4)$$

and set every $n_j = t$. Then $b_i + ts_i \geq c_i$ in every deficient coordinate, and nonnegative rays cannot spoil the other coordinates. This constructs a member of $X \cap \uparrow c$. With no rays, absence of an obstruction simply means $b \geq c$, and the parameter tuple is empty. $\square$

Different thresholds may use different obstructing coordinates. To show $X \cap \uparrow C = \emptyset$, the checker verifies the lemma separately for every $c \in C$; it need not find one coordinate separating the entire union. Likewise the common value t is merely a convenient witness. It is not claimed to minimize any initial population.

This interface accommodates an arbitrary number of waiting clients while keeping the checker entirely within exact natural arithmetic. It does not turn arbitrary parameterized programs into counter systems; population abstraction still needs its own source-simulation proof.

2.4 Search by minimal unsafe configurations

Let $U^{(0)} = \text{Bad}$ and repeatedly add all one-step predecessors. Each set is upward-closed, so store only its componentwise minimal vectors at each control. The producer keeps a queue of newly admitted vectors and an immutable justification node for each admission.

```

active[q] := empty antichain for every q
history := immutable nodes; pending := FIFO queue

admit(q, v, justification):
  if some w in active[q] has w <= v: return
  remove every w in active[q] with v <= w
  append (q, v, justification) to history
  put the node in active[q] and pending
  if an initial family at q covers v:
    construct its parameters and follow history to a bad node
    return the executable transition sequence

admit every target threshold, with terminal justification
while pending is not empty:
  remove a node (q', b) from pending
  if it is no longer active: continue
  for every incoming transition e : q -> q':
    admit(q, consume[e] + positive_part(b - produce[e]),
         (e, successor_node))
return the per-control active vectors

```

Removing a dominated vector does not remove its upward cone from the represented set: the newly admitted smaller vector covers it. Skipping its queued expansion is also sound. If $b' \leq b$, then $\text{pred}_e(b') \leq \text{pred}_e(b)$, so expansion of the smaller active threshold eventually covers the predecessors of the removed one. The worklist and active-map checks implement this argument rather than merely asserting that subsumption should be safe.

Theorem 2.4 (Termination and decision on the admitted fragment). *Without operational cutoffs, the search terminates. It returns either a backward-closed unsafe-region receipt disjoint from every initial family, or an executable initial-family instance and finite trace reaching Bad.*

Proof. Order $Q \times \mathbb{N}^d$ by equal control and componentwise comparison. Finite products of $\mathbb{N}$ are well quasi-ordered (Dickson's lemma), as is a finite disjoint union of them [6, 7, 9]. Suppose infinitely many vectors were admitted. At its admission each new vector is outside the upward closure of all earlier admitted vectors: deletion by subsumption does not change that accumulated upward region. Hence there are no earlier and later admissions in the same control with the earlier vector below the later one. This contradicts the well-quasi-order property. Each admission schedules only finitely many predecessor operations, so the worklist empties.

Initially the represented set is Bad. Every admitted predecessor can reach a previously justified upward cone, hence eventually Bad, by Lemma 2.1. Conversely at termination the represented set contains Bad and is predecessor-closed, so induction on trace length shows that it contains every state from which Bad is reachable. It is therefore the exact backward reachable region. Lemma 2.3 decides whether any initial family intersects it and constructs an initial instance whenever it does. The stored derivation supplies the corresponding trace. $\square$

The theorem does not imply a small basis, short runtime, short counterexample, or polynomial bit complexity. The mathematical termination argument and the engineering resource limit have different roles. The prototype reports `unknown` when its admission or predecessor budget is exhausted. A partially saturated basis is not a safe receipt.

2.5 A positive certificate smaller than the search

A positive receipt supplies finite lists C_q ; they need not be minimal, antichains, or search-generated. The checker verifies:

$$\forall b \in B_q, \quad \exists c \in C_q, \quad c \leq b, \quad (5)$$

$$\forall e : q \rightarrow q', \quad \forall b \in C_{q'}, \quad \exists c \in C_q, \quad c \leq \text{pred}_e(b), \quad (6)$$

$$\forall f, \quad X_f \cap \uparrow C_{q_f} = \emptyset. \quad (7)$$

The final condition uses the obstruction coordinates of Lemma 2.3. The certificate can carry those coordinates as indices for faster checking; the delivered simple checker finds them itself.

Theorem 2.5 (Counter receipt soundness). *Conditions (5)–(7) imply that no finite execution from an admitted initial state reaches Bad.*

Proof. Set $I(q, x) : \iff x \notin \uparrow C_q$. This holds initially. Suppose a valid step from (q, x) reaches (q', y) and $y \in \uparrow C_{q'}$. Choose $b \in C_{q'}$ with $b \leq y$. Lemma 2.1 yields $\text{pred}_e(b) \leq x$; (6) then gives a $c \in C_q$ below x , contradicting I . Thus I is preserved. By (5), I excludes every bad state. Induct over the finite execution. $\square$

This theorem needs neither Dickson's lemma nor correctness of the search. The formalized checker can be shipped before a formal proof of search termination. Its invariant is the *complement* of a backward unsafe region, which is often a disjunction:

$$I(q, x) \equiv \bigwedge_{c \in C_q} \bigvee_{i < d} x_i < c_i.$$

For natural counters, zero thresholds contribute impossible disjuncts; no subtraction of one from zero should be performed when presenting this formula.

If $M = \sum_q |C_q|$, the naive closure check costs

$$O\left(d \sum_{e:q \rightarrow q'} |C_q| |C_{q'}|\right)$$

integer comparisons and additions. Initial checks cost $O(d \sum_f m_f |C_{q_f}|)$ with a small extra term for the bases; precomputing which coordinates have nonzero ray sums reduces repeated work. All figures count arithmetic operations, not bit operations. Counter magnitudes and certificate byte limits remain explicit resources.

2.6 Why witness history must outlive subsumption

An unsafe receipt contains the selected initial family, its parameter tuple, and a list of original transition indices. The checker executes that list, checking every source control and consumption bound, and tests the final marking against the original bad set. It does not accept a producer's claim that a basis vector is reachable.

A backward justification has the shape

$$(q, \text{pred}_e(b)) \xRightarrow{e} (q', b) \xRightarrow{\text{suffix}} \text{Bad},$$

where the first arrow means that firing from the threshold produces a marking *at least* b , not necessarily equal to it. Monotonicity makes the same suffix executable from any larger marking. This is the reason threshold justifications can be composed.

The chain must use persistent node identifiers, not references to mutable antichain entries. In the one-counter fixture with increment $+1$, initial zero, and target 101, the active basis always has size one: 101 is replaced by 100, then by 99, and so on. The successful negative receipt nevertheless has 101 steps and uses 102 historical admissions. Deleting dominated history would destroy the only explanation of the answer. No shortest-counterexample claim is made for this subsumption search.

2.7 Worked unbounded examples

A nonconvex safe region. Start from any marking $(1+n, 0)$ or $(0, 1+n)$, with $n \in \mathbb{N}$. A transition consumes one token from a coordinate and returns two to that same coordinate. The unsafe target is $(1, 1)$. The single basis vector $C = \{(1, 1)\}$ is backward-closed, and each initial family has a different fixed zero coordinate. The receipt proves that both counters are never simultaneously positive, although either counter is unbounded. A single convex region containing both families cannot separate the bad target: it contains $(2, 0)$ and $(0, 2)$ and therefore their midpoint $(1, 1)$. This distinguishes the receipt from a single polyhedral invariant, not from every possible polynomial or disjunctive proof.

Mutual exclusion for arbitrary waiting populations. Use coordinates (ℓ, a, b, u, v) for a lock, active clients of two kinds, and waiting clients. The initial family is

$$(1, 0, 0, n, m), \quad n, m \in \mathbb{N}.$$

Entering the first critical section consumes one lock and one waiting token of kind u , and produces an active- a token. Release reverses this transfer; kind v acts analogously. The bad threshold is $(0, 1, 1, 0, 0)$. The delivered search computes six minimal backward thresholds, checks 24 predecessor operations, and certifies all n, m simultaneously. Removing lock consumption from the second entry rule produces a two-step trace at $(n, m) = (1, 1)$.

This toy protocol also has an obvious resource invariant $\ell + a + b = 1$ in its correct version. The experiment is not evidence that other methods cannot prove it; it demonstrates parameter extraction, original-edge replay, and a useful complete fragment without a user-supplied invariant.

3 Equality orbits instead of a finite test alphabet

3.1 The distinction that finite tests miss

A cache can have only two storage slots and still process arbitrarily many different keys. Its finite memory does not imply a finite concrete state space: the slots may contain any two atoms. Testing all request sequences over two chosen keys also does not represent all behaviors. It removes the possibility of receiving a third key while both slots are occupied.

The appropriate finite state is an *equality pattern*, including the relationships between both implementations being compared. Renamings of atom identities preserve execution because the allowed operations cannot inspect anything except equality. The general perspective belongs to nominal automata and permutation symmetries [8]. This section gives a direct, restricted construction whose entire search and replay are included.

3.2 A small, typed machine language

Fix an infinite set $\mathbb{A}$ of atoms, a finite tag set T , and distinct fixed constants $c_0, \dots, c_{c-1} \in \mathbb{A}$. A machine has a finite control q and k optional registers, each containing either an atom or $\perp$ (empty). An input is $(t, u) \in T \times \mathbb{A}$; $\perp$ is *not* an input atom.

A selector is one of: current input, a specified old register, a specified fixed constant, or empty. Guards are Boolean combinations of equality between selectors. Register updates simultaneously assign one selector to each register. An output is either a Boolean guard value or an optional atom selected from the same old-state/input environment. The output constructors are tagged, so Boolean false and atom zero cannot accidentally compare equal.

At each control and tag, the finite rule list is ordered. The first enabled rule is chosen. The admitted format requires an unconditional default rule for each control/tag cell; this makes execution total by construction. Rules after a default in the same cell are simply unreachable under this semantics. A consumer must not reorder rules or reinterpret the list as nondeterministic. Initial registers contain only constants or empty. Extra fixed initial atoms can be represented by adding constants, provided their distinctness and their source interpretation are proved.

The executable representation uses nonnegative integers as *names* for atoms, but no arithmetic or order operation on those integers is in the machine language. A Python sorting operation used to choose representatives is not an operation available to the modeled program.

3.3 Joint support and canonical representatives

For machines A and B with k_A and k_B registers, form the paired state

$$s = (q_A, q_B, r_A, r_B).$$

Let $k = k_A + k_B$. Its support consists of the distinct nonconstant atoms in the concatenated tuple (r_A, r_B) ; the fixed constants are retained separately. Two paired states are equivalent when their controls match and a bijection of atoms fixing every constant carries one *joint* register tuple to the other. Empty is also fixed.

For example, with no constants,

$$((a, b), (b, a)) \quad \text{and} \quad ((17, 6), (6, 17))$$

represent the same paired orbit. The pairs $((a), (a))$ and $((a), (b))$ with $a \neq b$ do not. Canonicalizing each machine's single register separately would map both pairs to $((0), (0))$, losing the exact fact that their outputs might disagree. The canonicalizer must see both states at once.

The producer preserves the integer labels of constants and scans the joint tuple left-to-right, assigning consecutive fresh labels to previously unseen nonconstant atoms. Empty remains empty. Canonicalization is a search optimization; the checker uses a different representation: the full pairwise equality matrix of

$$(\perp, c_0, \dots, c_{c-1}, r_{A,0}, \dots, r_{B,k_B-1}).$$

It therefore accepts noncanonical labels while preserving all cross-machine relations and all anchors.

Lemma 3.1 (Equality patterns characterize orbits). *Two optional-register tuples have the same equality matrix with the same fixed anchors if and only if a permutation of $\mathbb{A}$ fixing those anchors carries one tuple to the other.*

Proof. A permutation preserves equality, constants, and empty, proving one direction. In the other direction, equality of matrices defines a well-defined bijection between the finitely many nonconstant atoms appearing in the tuples. It maps each left occurrence to the corresponding right occurrence. A bijection between two finite subsets extends to a permutation of their union, and hence to a permutation of all atoms by fixing the complement. Constants remain fixed because their equality classes were anchored separately. $\square$

3.4 Why one fresh representative is enough

Given a paired state, enumerate as possible input atoms each live atom in either machine, each fixed constant, and one atom distinct from all of them. Tags are enumerated independently.

Lemma 3.2 (Fresh-input coverage). *For a fixed paired state, every input atom belongs to exactly one of the following relevant cases: a particular fixed/live atom, or the single class of atoms outside the fixed/live support. Every atom in the last class produces a successor and output pair related by a permutation fixing the current state.*

Proof. Two atoms outside the support can be exchanged while fixing the support and constants. By induction on selector and guard syntax, evaluation commutes with such permutations: equality is preserved, Boolean connectives are unchanged, and a selector either returns an anchored old value or the renamed input. The first enabled rule is therefore unchanged. Simultaneous copying and selected outputs commute with the permutation as well. Hence one chosen fresh atom represents all outside-support atoms. $\square$

“Fresh” here means absent from *currently remembered* registers and fixed constants, not never used at any earlier point in the execution. The machine cannot distinguish an old forgotten name from a globally new name. Insisting on globally new representatives would enlarge witness alphabets unnecessarily; merging a live name into the fresh case would be unsound.

For atom-valued outputs, compare the two actual selected values under the same representative input *before* forgetting their names. The property being proved is equality of the two outputs at each time, not a separate renaming of each output stream. No history of previous outputs is required for pointwise equality. A specification that tests equality with an arbitrarily old output would need to represent that additional memory explicitly.

3.5 Search and finite certificates

The producer performs breadth-first search from the paired initial orbit. For every stored representative, every tag, and every representative atom, it computes both outputs and both successors. Different outputs yield a counterexample. Otherwise it inserts the successor's *joint* canonical state. A parent pointer records the input case and predecessor for each newly visited orbit.

```

seen := { joint_initial_orbit }
pending := FIFO queue containing it
while pending is not empty:
  s := pop_front(pending)
  for tag in all_tags:
    for a in constants_and_live_atoms(s) + [one_fresh_atom(s)]:
      (outA, nextA) := execute(A, left(s), tag, a)
      (outB, nextB) := execute(B, right(s), tag, a)
      if outA != outB:
        lift_parent_path_to_concrete_inputs(s, tag, a)
        return Different(word)
      t := joint_canonical(nextA, nextB)
      if t not in seen:
        record its parent; add t to seen and pending
return Equivalent(list_of_seen_representatives)

```

A positive receipt is simply a finite list S of paired orbit representatives. The checker independently verifies that the initial equality pattern belongs to S , every representative input produces equal outputs, and every successor pattern belongs to S . It does not trust producer-supplied edge lists, a canonicalization hash, a minimality claim, or a search status.

Theorem 3.3 (Orbit receipt soundness). *An accepted positive receipt proves that A and B emit identical output sequences for every finite input word over $T \times \mathbb{A}$.*

Proof. Relate concrete paired states to the listed representative with the same controls and anchored equality matrix. The initial pair is related. For a concrete input, use Lemmas 3.1 and 3.2 to transport the pair and input to a checked representative case. That case has equal outputs and a successor represented in S . Transporting back preserves output equality and the successor relation. Induction on word length proves pointwise equality of the entire output sequences. $\square$

A negative receipt is a word of ordinary tags and concrete nonnegative atom labels. The independent checker executes the original two machines and finds an output mismatch. During witness construction the producer lifts each abstract input relative to the *current concrete joint state*; it does not concatenate canonical names from different states as though they were globally consistent identities. The latter shortcut is a common incorrect use of quotient paths.

3.6 Termination, cutoff, and witness length

Let B_n denote the n th Bell number, the number of partitions of n labeled positions. A tuple of k optional registers with c fixed constants is determined by a partition of $k + c + 1$ positions, with the anchor positions kept pairwise distinct. Thus B_{k+c+1} is a simple upper bound; it deliberately counts some impossible anchor identifications. With $c = 0$, the empty anchor makes B_{k+1} the exact number of optional equality patterns available over an infinite atom set.

Theorem 3.4 (Finite orbit decision). *Without cutoffs, the breadth-first search terminates after visiting at most*

$$N = |Q_A| |Q_B| B_{k_A+k_B+c+1}$$

paired orbits. It either returns a positive receipt or a shortest differing input word. If R paired orbits are reachable, a differing word, when one exists, has length at most R .

Proof. There are at most N control-pattern pairs, each with finitely many input cases. If all are exhausted without a mismatch, the visited set satisfies Theorem 3.3. If a mismatch is found, path lifting gives a concrete differing word. Conversely every concrete path projects to this orbit graph, so a concrete mismatch cannot be missed.

An output mismatch occurs on an outgoing edge. A shortest path to the source of such an edge has no repeated paired orbit: a repeated segment can be removed, and the remaining abstract path can still be lifted. The prefix has length at most $R - 1$, giving total length at most R . Breadth-first traversal considers all shorter prefixes first, so its first mismatch has minimum word length. $\square$

This is not a claim that the implementation visits every orbit up to the Bell-number bound. Often almost all are unreachable. The cache-renaming example needs only three paired orbits despite the crude bound $B_5 = 52$.

Corollary 3.5 (A justified finite atom carrier). *Let $F \subseteq \mathbb{A}$ contain all fixed constants and have size at least $k_A + k_B + c + 1$. The two admitted machines are equivalent over $\mathbb{A}$ if and only if they are equivalent on all words over $T \times F$.*

Proof. One implication is restriction. For the other, lift any differing orbit path using names in F . At each point at most $k_A + k_B$ nonconstant names are live, so F contains a name outside the live/fixed support. Use that name for the fresh case, reusing names only after they have been forgotten. This preserves every required equality and inequality along the path and produces a differing word over F . $\square$

The result is a completeness theorem for *all words over a derived carrier*, not for all words up to a chosen length. It also assumes the specified copy/equality language and anchored initial registers. It does not justify a similar cutoff for arbitrary programs that compare keys by hash or order, remember an unbounded set, or create fresh atoms internally with additional semantics.

3.7 Worked example: the cache distinction

The fixtures compare a two-slot least-recently-used cache with a first-in, first-out cache. Both start empty, both report whether an input key was present before the request, and both insert a missed key at the front while dropping the old tail. On a hit in the tail, LRU moves the hit to the front while FIFO leaves the insertion order unchanged. Head hits leave both unchanged.

The search returns the word

0, 1, 0, 2, 0.

The first two requests fill the cache. The third makes 0 most recent under LRU, but does not change FIFO's insertion order. Request 2 then evicts 1 under LRU and 0 under FIFO. The final request is a hit only under LRU. The delivered search reports five visited pair orbits and eleven checked representative edges before finding the mismatch.

Request	0	1	0	2	0
LRU hit output	false	false	true	false	true
FIFO hit output	false	false	true	false	false

Exhaustive concrete exploration over *only two keys* visits seven paired states and reports equivalence. That answer is correct for that smaller alphabet: after both keys have appeared,

all later requests hit. It is wrong as a conclusion about arbitrary keys. The fresh representative is the missing semantic case, not merely a request for a larger random sample.

As a positive control, swapping the two physical registers and consistently rewriting all selector indices produces an equivalent implementation. Its certificate has three joint representatives:

$$(\perp, \perp \mid \perp, \perp), \quad (0, \perp \mid \perp, 0), \quad (0, 1 \mid 1, 0).$$

There are six representative input checks. The same three-state relation also certifies a variant that returns the evicted atom rather than a hit Boolean. This exercises cross-machine equality, empty registers, and atom-valued outputs, not just control-state isomorphism.

3.8 Checking cost and representation choices

Let L bound the cost of interpreting the rule list and expressions for one transition, $k = k_A + k_B$, and R be the receipt's orbit count. Each orbit has at most $c + k + 1$ representative atoms per tag. With hashed equality signatures, the checker costs roughly

$$O(R|T|(c + k + 1)(L + (c + k)^2))$$

primitive operations, plus input validation. The quadratic term is the simple pairwise equality-matrix construction, intentionally distinct from the producer's linear scan canonicalizer. A verified partition encoding could reduce it later. Bell-number growth remains the limiting factor as register count rises; the construction is not advertised as a general scalable replacement for every symbolic automata algorithm.

4 Names and counters in one exact worker

4.1 The admitted mixed language

The mixed state is (q, r, x) , where q is finite control, r is a tuple of k optional equality registers, and $x \in \mathbb{N}^d$ is a counter vector. A rule contains a source and destination control, an input tag, an equality guard over old registers and the input, simultaneous register-copy updates, and fixed consumption/production vectors (a, z) .

Unlike the deterministic equivalence worker, the mixed language is *nondeterministic*: every rule whose tag, guard, and consumption bound hold is available. It need not have a default, and it can deadlock. The target is universal safety over all finite rule/input executions; no fairness or eventual release property is asserted. In particular a blocked execution cannot refute safety merely because it stops making progress.

A bad clause combines a control location, a guard over registers and constants only, and an upward counter threshold. The current input is forbidden in bad clauses because a state property cannot depend on an unspecified next input. Initial registers are anchored, and counter initial states form the families of (2). The current prototype gives these families a common initial control and register tuple; finite unions of different anchored starts would be a straightforward extension of the representation but are not claimed as implemented.

4.2 Compile structural name orbits first

Explore the finite orbit graph of (q, r) from the anchored initial tuple, considering every rule and every relevant atom representative. During this *structural* exploration, ignore counter enabledness. An edge records the original rule identifier, tag, representative atom, and the exact vectors a, z . The resulting finite graph is a finite-control counter system.

Ignoring counters here does not mean deleting their guards. It only discovers an overlarge finite set of possible name-control locations. The compiled counter transition still requires

$x \geq a$. A structurally discovered location may be unreachable with any feasible counters; retaining it cannot invent a feasible run because the counter semantics on its incoming edges remains exact.

Theorem 4.1 (Exact orbit-counter transport). *For the admitted mixed language, every concrete execution projects to a path of the compiled finite-control counter system with exactly the same counter vectors. Conversely every compiled path from an initial family instance lifts to a concrete mixed execution with the same counters. The bad-state predicates are preserved. Hence mixed safety is equivalent to coverability safety of the compiled system.*

Proof. For projection, normalize the name tuple after each concrete step. The input is represented by a fixed/live case or the fresh case; guard evaluation and register updates commute with the corresponding permutation. That rule edge was therefore enumerated by structural exploration. Its vectors a, z are unchanged, so enabledness and counter successors agree exactly.

For lifting, proceed along the compiled path. Maintain a bijection between the current representative's live names and the concrete live names, fixing constants. Realize the recorded input as its mapped live/constant name or as a name outside the concrete support. The equality guard is true, the register update reaches the required orbit, and the path's counter enabledness makes the original rule executable. Repeat. The name guard in a bad clause is invariant under these renamings, and its counter threshold is untouched. $\square$

The theorem depends on the separation of equality syntax and counter updates. It says nothing about an unbounded dictionary $\mathbb{A} \rightarrow \mathbb{N}$ of per-name resources, atom ordering, or guards that compare counters with atom identities. It also does not establish a closure theorem for arbitrary products of infinite-state decision procedures. It proves a specific transition correspondence for a specific language.

4.3 Check against the mixed source, not the compiler output

A positive receipt supplies a list of structural orbit representatives and an unsafe counter basis for each. The independent checker verifies the initial orbit's presence and initial-family separation. It then independently evaluates every original rule on every representative atom. For each enabled equality case it requires the successor orbit to be listed and checks (6) with that rule's original consumption/production vectors. It also checks inclusion of every bad threshold whose original register guard holds at the representative.

Thus a bug that omits an edge during structural compilation cannot hide behind a successful counter proof for an incomplete graph. The receipt is replayed from the caller-supplied mixed model, and its graph is reconstructed by the checker using equality matrices, not the producer's canonicalizer. This is still two Python implementations, not a formal verification result, but the separation targets the actual new compiler boundary.

The positive proof combines forward closure of the finite name relation with backward closure of the per-orbit unsafe region. Induction maintains

$$\text{the current name orbit is listed} \quad \wedge \quad x \notin \uparrow C_{\text{current orbit}}.$$

The first conjunct comes from the name edge check, and the second from Theorem 2.5's local predecessor argument. A negative receipt contains initial-family parameters and triples `[ruleId, tag, concreteAtom]`; the checker executes the original mixed rules and counter consumption checks directly.

4.4 Worked example: a remembered owner and arbitrary clients

Use one optional owner register and counters (ℓ, a, w) for free lock tokens, active leases, and waiting clients. Initially

$$r = \perp, \quad (\ell, a, w) = (1, 0, n), \quad n \in \mathbb{N}.$$

An `acquire(u)` rule requires $u \neq r$, consumes one free lock and one waiting token, produces one active lease, and remembers u as owner. A `release(u)` rule requires $u = r$, consumes an active lease, produces a free lock and a waiting token, and empties the owner register. The safety target is $a < 2$ at every finite execution prefix. The model is a deliberately small resource/identity protocol, not a complete implementation of a multi-client lock service.

There are only two structural name orbits: empty owner and some remembered owner. Their compiled graph has three edges: acquisition from empty, acquisition with a fresh relative owner, and matching-owner release. The second acquisition edge is often disabled by the counters; retaining it is required for faithful structural compilation and for checking faulty versions.

The positive receipt uses the same three backward thresholds at both orbits:

$$C = \{(0, 2, 0), (1, 1, 1), (2, 0, 2)\}.$$

Initially, the first two thresholds are blocked by $a = 0$, and the third by $\ell = 1 < 2$, independent of n . The full receipt therefore has two name representatives and six stored threshold vectors. The producer performs nine predecessor operations.

Now omit lock consumption from acquisition. The worker chooses $n = 2$ and returns

$$\text{acquire}(0), \quad \text{acquire}(1).$$

The two names must differ because the second guard compares with the remembered owner. Counter replay yields active count two. Reusing a single representative atom for all requests would suppress this counterexample. Abstracting away counters without retaining consumption would instead falsely reject the correct version. This example exercises both sides of the product theorem.

4.5 A finite oracle for differential testing, not for proving this case

For randomized mixed tests the generator conserves total counter mass and uses one fixed initial marking. Its concrete counter state space is then finite. Combining that finite set with an atom carrier of size $k + c + 1$ gives an exhaustive concrete oracle for the generated instances. This oracle does not build name orbits or antichains. It is suitable for differential testing of the algorithms.

The parameterized owner example is not proved by that finite oracle: its initial family has unbounded mass. Its delivered evidence is a checked symbolic receipt under the mathematical arguments above. Keeping these two evidence sources separate avoids silently converting a test on finitely many clients into an all-populations theorem.

5 Lean integration at the actual Forge boundary

5.1 Reuse the existing contracts

Forge already separates logical certificate specifications from runtime candidates. Its `ForgeContracts.CertificateSpec` has `denotes`, `check`, and a theorem `sound`; its runtime sketch provides `ForgeDesign.Obligation`, `Candidate`, `ConditionalCandidate`, `ScopeKey`, and

`EngineReply` [2, 3]. The new workers should implement instances of those contracts, not introduce another independent authority or scheduler.

For the counter worker, the `denotes` proposition says every admitted initial state avoids the supplied upward bad set along every finite execution. For the nominal worker it says the two supplied model interpretations have equal output lists for every input list. For the mixed worker it says the original register-net model is safe, not merely its producer-generated graph. The model is supplied by the caller. A certificate must not replace it with an easier model carrying a similar name or an identical debug string.

A candidate accepted by the logical checker is only one part of closing the source goal. The source bridge must establish the precise proposition needed by the current obligation, including its types, local assumptions, and observations. Forge’s existing scope and candidate-publication rules remain in force. A JSON field declaring `safe` is not a `Lean.Expr` proof.

5.2 A proof-carrying frontend for counter programs

Let a source state space be S , source transition relation be $\rightarrow_S$, and source initial predicate be Init_S . A safe abstraction into a model M requires a map $\alpha : S \rightarrow Q \times \mathbb{N}^d$ and three bridges:

$$\text{Init}_S(s) \implies \alpha(s) \in \text{Init}_M, \quad (8)$$

$$s \rightarrow_S s' \implies \alpha(s) \rightarrow_M^* \alpha(s'), \quad (9)$$

$$\text{Bad}_S(s) \implies \text{Bad}_M(\alpha(s)). \quad (10)$$

For the first implementation, require one model step per source step; permitting finite stuttering paths is a later representation choice with an explicit path proof. These conditions let a model safety theorem imply source safety. The direction of (10) is essential: excluding a subset of the source bad behavior would not establish the required goal.

The converse direction is needed for a source-level counterexample. A model trace can be spurious under an overapproximating abstraction. Publish a negative result as a refutation of the *model* unless every edge has a source lift and the initial and bad endpoints are connected appropriately. For an exact reification of the admitted rule language, the lift is available by evaluation; for a manually supplied abstraction it may not be. The runtime should then return a diagnostic or a refinement obligation, not falsely assert the original Lean proposition has been disproved.

A restricted frontend can inspect a function that selects a finite transition and updates a fixed tuple of natural counters. It should recognize checked predicates of the form `consume[i] <= x[i]` and expressions `x[i] - consume[i] + produce[i]`. It must prove, rather than assume, that the enabled branch is used. Lean’s natural subtraction is truncated; the unconditional expression may be defined even when the model’s consumption transition is disabled. The bridge therefore exposes the guard as a hypothesis of each transition theorem.

The local mathematical core is small:

```
-- Proposed local proof obligations; not an installed Forge API.
preScalar a z b := a + (b - z) -- Nat subtraction

preScalar a z b <= x <->
  a <= x /\ b <= x - a + z

preVector consume produce b <= x <->
  consume <= x /\ b <= stepCounters consume produce x
```

The shipped `lean/CounterLemmas.lean` spells out these facts and the one-step complement-invariant argument using core Lean and `omega`. It has no placeholders, but it was not compiled

here. It is not the missing certificate-soundness theorem and does not make the Python checker trusted.

5.3 A proof-carrying frontend for equality programs

For a deterministic source transducer with state S , one-step function

$$\text{step}_S : S \rightarrow T \times \mathbb{A} \rightarrow O \times S,$$

a model registration should provide a relation $R : S \rightarrow \text{ModelState} \rightarrow \text{Prop}$ with an initial proof and a step diagram. If $R(s, m)$, source and model steps on the same input must give related successor states and equal observations. Two such registrations transport a model equivalence certificate to equality of source output lists by induction. A relation, rather than only a projection, accommodates different physical register layouts and hidden source fields.

The one-step bridge can often be generated by splitting equality guards and simplifying selector updates. `grind` or `simp` can help discharge these *local* equations, but the all-input theorem comes from the checked orbit certificate. That is the division of responsibility: local reasoning establishes that a source step has the model semantics, and the worker settles an unbounded family of steps.

In Lean, equality of keys must really mean equality. For `BEq`-based source code, request a law connecting the Boolean result with propositional equality, such as a suitable `LawfulBEq` assumption or an explicit theorem for the comparison function. A hash-only comparator does not become suitable merely because its type is `Alpha -> Alpha -> Bool`. The frontend should reject ordering, hashing, conversion to integers, or arbitrary user callbacks unless a separate proof reduces their behavior to the admitted equality language.

An arbitrary `DecidableEq` dictionary supplies equality tests, not additional observations about the representation of keys. The source statement should make this semantic scope clear. For a finite key type too small to realize every proposed fresh case, the infinite-atom model is a conservative extension for *positive* equivalence: its theorem restricts to the finite type. A negative word must be checked for realizability in that type. For an infinite key type, a finite set of distinct witness labels can be embedded; a concrete witness printer can use the source type's supplied sample constructor only after validating distinctness.

5.4 Registering summaries of reusable primitives

The useful library unit is a proved *one-step summary*, not an unchecked string describing a function's behavior. For example, a two-entry cache primitive can register a model together with the diagram connecting its record fields, equality tests, and return value to that model. A register permutation can be composed into that relation without resynthesizing an entire proof about arbitrary input lists.

Proposed module boundaries, not existing declarations, are:

```
Forge/Infinite/Counter/Model.lean
Forge/Infinite/Counter/Certificate.lean
Forge/Infinite/Counter/Soundness.lean
Forge/Infinite/Equality/Language.lean
Forge/Infinite/Equality/Patterns.lean
Forge/Infinite/Equality/Certificate.lean
Forge/Infinite/Equality/Soundness.lean
Forge/Infinite/Mixed/Model.lean
Forge/Infinite/Mixed/Soundness.lean
Forge/Infinite/Frontend/Counter.lean
Forge/Infinite/Frontend/Equality.lean
Forge/Infinite/Worker.lean
```

The model and certificate modules should not import the search implementations. The equality-language interpreter should be the common *formal semantics*, while the search remains untrusted. The Python choice of different producer and checker interpreters is a testing device, not a recommendation to duplicate the trusted Lean semantics forever.

5.5 Candidate construction and proof replay

The worker flow is concrete. Recognize an admitted source obligation or find a registered summary. Build the model together with its bridge proof. Submit only the model data to the external or native search. Parse the returned receipt with resource limits. Compute the checker result and construct a proof of its truth using kernel-supported evaluation or proved reflection. Apply the corresponding `CertificateSpec.sound` theorem, then the source bridge. Finally publish a `Candidate` for the original obligation in its original scope.

When some bridge facts are missing, return a `ConditionalCandidate` whose premises are those exact obligations and whose constructor uses *all* of them. Do not publish the model theorem as if it had closed the source goal. An unsupported atom primitive maps to `FailureKind.unsupported`; a search cutoff to `budgetExhausted`; a malformed or rejected receipt to `rejectedEvidence`. These are existing Forge failure categories, not a new mutually incompatible result protocol.

The equality worker’s successful result is an equivalence theorem; the counter worker’s is a safety theorem. Routing should use the goal’s semantic shape and available bridge, not just names such as `cache` or `lock`. A counterexample from a candidate checker is useful search feedback, but the boundary must keep proof goals, data synthesis, and model diagnostics distinct.

5.6 How Leant and Djex fit, without overclaiming their problems

Leant’s current indexing-composition report is especially instructive: the required function-valued fold and the integer-case step appear separately, while the original bounded composition query still has no accepted implementation. The report distinguishes an isolated step’s candidate ordinal from its position in the original query and preserves the original failed cell [4]. Djex likewise distinguishes checked higher-rank synthesis from completeness only in narrower fragments [5].

The new workers should therefore be optional *semantic discharge services* after normal candidate construction and reconstruction, not a replacement for lexical identity, universe selections, dictionary evidence, or compiler checks. For a candidate and specification with proved equality-machine summaries, run exact orbit equivalence. An inequivalent candidate yields a concrete input word for the behavioral filter; an equivalent candidate yields a finite receipt that can be turned into an all-input proof once the Lean checker and source bridge exist. For a resource-managing candidate with a proved mixed summary, use the mixed safety worker analogously.

The curated Djex embedding entry point is `Language.Haskell.Djex`, with narrower checked adapters or focused `Language.Haskell.Synthesis.*` modules for specialized use [5]. An integration should consume the reconstructed candidate’s identity and attached, proved semantic summary, not private search-node structure. A cache of validated model equivalences must include the exact model, fixed constants, source bridge, interpretation of outputs, and environment identity; reusing a certificate solely by rendered source name is insufficient.

No current prototype extracts a register machine from a higher-rank term, compiles Church encodings, or closes Leant’s original integer-indexing query. Nor does this article present continuation-local synthesis as a new discovery; Forge’s extension ledger already contains it.

The added capability is exact all-word or all-population validation for candidates whose finite semantic interface has actually been established.

6 Executed evidence and its limits

6.1 What was run

The delivered implementation uses only the Python standard library and was run under Python 3.13.5 on Linux. The test command uses `python -S`, disabling site-package initialization. No SMT solver, numerical library, or external model checker was installed or used. The original corpus and its exact models, receipts, oracle outcomes, and recorded timings are in `results/`. The generator seed is 20260915.

There are three different units of evidence. The unit suite has 27 test methods, some containing many assertions. The randomized suite contains 640 model or model-pair instances. The stored replay corpus has those 640 instances plus ten handcrafted examples, for 650 certificate objects. The mutation suite makes two invalid edits to each of those 650 objects, for 1,300 rejection checks. Those numbers should not be added into a single success count.

Randomized family	Cases	Positive	Negative	Oracle agreement
Finite-control counters	300	222	78	300/300
Equality-machine pairs	160	123	37	160/160
Mixed register/counter systems	180	116	64	180/180

“Positive” means safe for the counter and mixed workers, and equivalent for the equality worker. “Negative” means unsafe or inequivalent, with an executable witness. No randomized case returned unknown at the configured limits. That is a property of these small generated corpora, not a general success-rate prediction.

For all 37 inequivalent randomized equality pairs, the returned word length also agreed with the exhaustive oracle’s shortest mismatch length. The other workers do not claim shortest witnesses. In particular, the common-parameter construction may choose a larger initial population than necessary.

6.2 What the differential oracles actually do

The random counter generator conserves total token mass and uses a fixed initial marking. The oracle exhausts the finite reachable concrete graph; it does not truncate counters at an arbitrary chosen value. Because the state space is finite for these inputs, its negative search result is a complete result for the generated model, not a depth-bounded observation.

The equality oracle performs concrete paired-state BFS over an atom carrier of size $k_A + k_B + c + 1$. Corollary 3.5 justifies that carrier for the admitted machines. It does not build a quotient or use the producer’s canonical forms. The random mixed oracle similarly combines conservation of token mass with a derived atom carrier of size $k + c + 1$.

These are algorithmically different oracles, but not wholly independent implementations of every layer: they reuse the checker’s concrete evaluator. The certificate checker itself does not import the producers and independently implements arithmetic predecessor checks and nominal interpretation. Thus the evidence detects many producer mistakes, including quotient and worklist mistakes, but a semantic bug shared by the checker and oracle can survive their agreement. The paper proofs and targeted tests reduce that risk; only a formalized soundness theorem would close the logical gap.

6.3 Workload size and construction bias

Across the randomized counter corpus the oracle visits 537 states; across the nominal corpus it visits 1,097, compared with 440 producer-reported orbit visits; across the mixed corpus it visits 479. These are intentionally tiny systems, often settled at or near the initial state. They are suitable regression and differential tests, not scalability evidence. The aggregate orbit/concrete counts also do not justify a speedup ratio: exploration can stop at different points, and the operations have different costs.

Nominal pairs are built from a random source machine and a register permutation. On alternating cases one output expression is changed. Some changes are on unreachable or observationally irrelevant branches, which is why the count of inequivalent pairs is less than half the corpus. This construction supplies both semantic positives and negatives, but is biased toward closely related machines. It is not an unbiased sample of program equivalence obligations.

Recorded single-run times for the randomized loops, including their oracle comparisons, are approximately 0.068 seconds, 0.159 seconds, and 0.076 seconds for the counter, nominal, and mixed families respectively. These values merely show that the experiments executed; there is no timing distribution, warm-up control, Lean elaboration cost, or comparison with a tactic. Reproduction on another interpreter may change the timings without changing any certificate.

6.4 Handcrafted results

Fixture	Result	Useful distinguishing evidence
<code>axes</code>	Safe	One backward vector for two nonconvex, unbounded initial families.
<code>mutex</code>	Safe	Six vectors; all two-parameter waiting populations; 24 predecessor operations.
<code>mutex_bug</code>	Unsafe	Two steps at parameters (1, 1); every consumption guard replayed.
<code>enabling</code>	Safe	A zero-net-change edge still requires one token; initial zero cannot fire.
<code>delayed_101</code>	Unsafe	101-step trace; 102 historical nodes despite active antichain size one.
<code>lru_register_swap</code>	Equivalent	Three paired orbits; six representative edges; joint equality preserved.
<code>lru_fifo</code>	Different	Five requests using three atoms. Two-atom exhaustive exploration is a deliberate inadequate-alphabet control.
<code>eviction_register_swap</code>	Equivalent	Same three orbits, but optional-atom outputs rather than only Booleans.
<code>owner_pool</code>	Safe	Two name orbits, six total counter vectors, unbounded initial waiting count.
<code>owner_pool_bug</code>	Unsafe	Two acquisitions with distinct names at initial waiting count two.

The finite two-key LRU/FIFO oracle result is stored alongside the examples but is not a separate certificate object. It records seven concrete pair states, no mismatch, and alphabet size two. Keeping its scope visible is the point of the control.

6.5 Mutations and positive robustness controls

The 1,300 corpus mutations are primarily *structural or required-content* checks: an invalid schema version, followed by a missing mandatory state/basis, an out-of-range initial-family index, or an empty mismatch word. Their complete rejection does not establish resistance to every semantic forgery or hostile input. The count should not be marketed as 1,300 deep independent attacks.

The unit suite adds targeted semantic controls. It exhausts 625 small scalar instances of the exact predecessor identity and checks 64 scalar initial-family separation instances. It tests enabling versus net change, a backward-closure failure even when target inclusion and initial separation pass, insufficient fresh-name alphabets, fixed constants, empty versus atom zero, source-model mutation, lost orbit states, truncated witnesses, and unknown budget outcomes. It also rejects malformed numeric values, wrong vector dimensions, unsupported atom operations, duplicate JSON keys, and nonfinite JSON constants.

Positive robustness controls matter too. A safe basis extended with redundant or dominated vectors is accepted when it still satisfies the obligations. Renaming every nonconstant atom label in an equivalence certificate to sparse, noncanonical integers is also accepted. A checker that rejects every edited receipt may simply be checking agreement with its producer, not the intended mathematical property.

6.6 Replay separation and remaining risks

The standalone replay command imports only the checker and standard-library modules; it asserts that the producer and oracle modules are absent from `sys.modules`. It accepts all 650 stored objects, distributed as 305 counter, 163 nominal, and 182 mixed objects. The checker’s fresh representative is chosen differently from the producer’s, and its equality pattern is a matrix rather than the producer’s scan-normal form.

These facts establish operational separation, not a trusted computing base. Python, the parser, the arithmetic implementation, the evaluator, and the checker remain unverified. The JSON loader limits total text size and rejects some malformed encodings, but it is not hardened against every adversarial resource-exhaustion pattern. In particular, large lists, huge integers, pathological expression trees, and expensive receipts need additional allocation, node-count, and bit-length budgets before use as an unattended service.

No source reifier, Lean checker, or all-input Lean theorem was executed here. The small Lean specimen has status `NOT_RUN`. The baseline repository’s previous Lean elaborations remain its own evidence and are not attributed to these new workers. The delivered results justify implementing and evaluating the formal workers; they do not establish that Forge has already gained any end-to-end theorem-solving capability.

7 Concrete libraries, engineering plan, and acceptance gates

7.1 Library choices and what not to outsource

The first implementation should keep the receipt checkers small. The counter checker needs finite lists, exact natural arithmetic, indexed vectors, and bounded iteration over source rules. The equality checker needs finite tagged syntax, its interpreter, equality-pattern construction, and finite membership. No general SMT solver is necessary for either receipt language.

For a Lean implementation, `Fin d -> Nat` is a useful mathematical counter representation; an array-backed executable form should be related to it by a proved length/index invariant. The distinction prevents mismatched dimensions from being silently truncated by a zipped traversal. `omega` is an appropriate way to establish the scalar consumption/predecessor lemmas during library development, while a proved Boolean checker can handle the many ground

instances by reflection. The worker should not construct a fresh, large `omega` proof for every coordinate of every stored threshold if a shared soundness theorem suffices.

For search termination, Mathlib’s documented `Mathlib.Order.WellQuasiOrder` provides `WellQuasiOrdered` and `WellQuasiOrdered.pi`, the finite-product theorem needed for Dickson’s argument. `WellQuasiOrderedLE.finite_of_isAntichain` supports a related finiteness statement [9]. The public documentation was checked, but its rolling build is not a proof that every spelling compiles at Forge’s exact Mathlib pin. Resolve and compile that pin during implementation. The delivered counter receipt soundness proof does not need this dependency: completeness of the search and soundness of finite receipts should remain separate modules.

The equality worker can initially use a direct finite-pattern proof rather than importing a broad nominal-sets framework. Define renaming action on selectors, guards, states, and outputs; prove evaluation equivariance by structural induction; prove equality-matrix completeness for finite support; then prove representative-input coverage. This keeps the trusted formal layer close to the implemented language. General permutation and finite-set support can be reused from Lean/Mathlib, but no unverified guessed API for a complete register-automata library is assumed here.

Component	Specific role	Qualification
Lean core and Mathlib	Exact arithmetic, finite structures, local guard proofs, WQO product theorem	The portable core checker is the priority. Formal source bridges and checker correctness remain to be written.
MinCoverPetri	Optional independent counter-model oracle or alternative producer	C++ tool for PNML P/T nets, using monotone-pruning coverability; GPLv3+. Its forward omega-cover is not our backward receipt.
RaLib	Optional generator of candidate register-automaton summaries	Java active-learning library associated with LearnLib. Learned summaries still need source equivalence proofs; learning output is not a receipt.
Existing Forge workers	Discharge source-side arithmetic or logical obligations	Reuse local <code>grind</code> /arithmetic support and the existing candidate contracts; do not implement a second proof-publication path.
Djex checked embedding	Obtain reconstructed candidates for exact behavioral discharge	Use the curated embedding layer and proved summaries, not an untyped mirror of private synthesis states.

MinCoverPetri’s documentation gives a particularly important adapter constraint: its documented default uses signed 8-bit marking values and reserves 127 for omega, interpreting initial markings at least 127 as omega. It can be rebuilt with different representation choices [10]. An adapter must enforce the accepted finite range or verify a suitable build before claiming agreement on an exact finite marking. It must also encode finite control without changing which transitions are enabled. Even with those repairs, the tool’s forward minimal-cover result is not a certificate for (5)–(7); a separate conversion theorem or producer is necessary. The delivered prototypes do not invoke it.

RaLib supplies active-learning algorithms for register automata [11]. That makes it potentially useful when exploring an interface for which an approximate model can be learned from queries. It does not remove the need for exact source-model correspondence. For the

admitted finite syntax, direct compilation is simpler and more trustworthy than learning a model already present in the source. No RaLib experiment is reported here.

7.2 Producer data structures

The prototype uses ordinary lists and dictionaries for clarity. A production counter producer should index incoming transitions by destination, retain a separate immutable justification arena, and keep per-control antichains in a dominance-search structure. For small d , sorted coordinate indexes can prune dominance candidates before exact comparison; for larger d , a trie or multi-index structure may help, but its usefulness must be measured on real receipt shapes. A list is an appropriate baseline, not an embarrassing implementation to replace without evidence.

Maintain antichain version numbers on queued nodes so obsolete entries can be skipped in constant time after a lookup. Record operations on exact integers; never convert thresholds to floating point to speed comparisons. If a heuristic or external tool produces a proposed subset of unsafe thresholds, the checker still requires full backward closure before using it as a safe receipt.

The nominal producer can hash canonical tuples with controls and anchored constant count. It must not use independent left/right hashes as if their combination preserved cross-state equalities. Cache interpreted transitions by model identity, control, joint pattern, tag, and input class. An optimization that reuses a single-machine transition table must transport atom outputs and register values back into the joint namespace before comparison.

For mixed search, retaining quotient edge labels is necessary for negative witness lifting. Deduplicating edges that share controls and vectors is safe for pure coverability, but a witness still needs at least one original rule/input-case justification. Dropping those labels turns a numeric path into an unexplained diagnostic.

7.3 A phased implementation with falsifiable gates

Gate 1: a counter receipt theorem. Implement the model, Boolean vector comparisons, and finite basis checks. Prove the exact predecessor lemma and `check_sound` for the supplied model. Make the one-vector axes receipt prove a source-level theorem for all initial parameters, with no synthesis during subsequent compilation. Record successful kernel elaboration and the actual axiom inventory. Accept a redundant valid basis and reject the enabling trap’s fake unsafe witness.

Gate 2: counter source reflection. Compile only the admitted guarded tuple-update language. Every rule receives a proved transition diagram. Add the parameterized mutex family and its bug, requiring the positive theorem for all waiting counts and a source-executable negative witness for the faulty version. Mutation of the original rule vectors must invalidate the retained receipt. A proof of the JSON model alone does not pass this gate.

Gate 3: equality-pattern correctness. Formalize the typed equality-language interpreter, joint pattern relation, anchored constants, and the fresh-input lemma. Prove the finite-carrier theorem with its register-count hypotheses. Compile the three-orbit register-swap receipt and the five-request LRU/FIFO witness. Include atom-output and insufficient-alphabet controls. A theorem merely about one selected finite alphabet does not pass.

Gate 4: mixed direct replay. Formalize the original mixed language and the two-component invariant. Check all original source rules rather than trusting a producer-emitted graph. Prove the parameterized owner-pool theorem and lift its faulty-version witness to two

actual source steps. Deliberately remove a structural quotient edge in a producer test and ensure direct replay rejects any resulting incomplete positive receipt.

Gate 5: controlled tactic evaluation. Only after the preceding kernel-checked paths work, compare against existing `grind`, appropriate induction/`simp`/`omega` baselines, and Forge’s nearest existing workers. Include strong manually supplied invariants as a separate condition, not a hidden advantage for one side. Keep model reflection, search, parsing, receipt checking, bridge construction, final kernel checking, and ordinary recompilation as separate timing categories. A theorem discovered once should recompile from its retained certificate without relaunching the producer.

A meaningful test set should include hand-written Lean source definitions, representation changes that need nontrivial bridges, small genuine counterexamples, source fragments that must be rejected, increasing register width, increasing counter dimension, and bases with many incomparable vectors. Neither conservation-biased random nets nor register permutations alone cover that range. Report unsupported and unknown outcomes separately from incorrect ones; a timeout must not become a refutation or a false positive.

7.4 Routing and useful refusal diagnostics

Routing is mechanical once a bridge exists. Upward counter safety with monotone consume/produce rules selects the antichain worker. Equality of output-list functions with finite-register summaries selects the orbit worker. A register-net safety bridge selects the product. If the semantic shape fails, return the exact obstruction to the planner: a zero-test, an order-sensitive key operation, an unbounded register allocation, an unsupported initial parameter constraint, or an output depending on absent history.

Some rejected shapes can become conditional problems. A zero-test may be eliminated after a proved invariant makes its outcome constant on a control location. A key comparator may have a supplied theorem showing it agrees with equality. A source collection may have a proved fixed-size summary sufficient for the observation. These transformations are useful, but their proofs must be explicit premises, not silent weakening by the worker.

7.5 A restrained direction beyond this delivery

The most attractive next extension is not to add another unrelated solver, but to make registered summaries compositional. A proved operation summary should survive register renaming, product state formation, and hiding unobserved fields. Forge’s existing observable quotients can then reduce a finite numeric summary after the new name quotient has been justified. One must not simply apply an ordinary finite-dimensional rank bound to a space indexed by infinitely many atoms. Weighted register automata have a separate orbit-finite-dimensional theory [12]; using it would require a new certificate language and proof, not a change to one dimension parameter. This is a research direction, not an implemented fourth worker.

The practical next action is Gate 1. These algorithms already run as research programs; the largest remaining gain in confidence comes from turning one exact receipt and one source bridge into a Lean theorem, not from increasing the number of uncompiled design contracts.

A Reproducing and using the artifacts

The package contains the article sources and compiled PDF, the three producers, a search-independent checker, exhaustive finite oracles, unit tests, recorded inputs and receipts, example JSON files, a small uncompiled Lean lemma specimen, and build/replay instructions. No

third-party source tree or font file is included. The Python tools require Python 3.10 or newer; only 3.13.5 was used for the recorded run.

From the package root:

```
python -S tests/test_core.py
python -S prototype/replay.py
python -S prototype/run_experiments.py --output reproduced-results

python -S prototype/forge_worker.py search vass \
  examples/mutex.model.json --output candidate.json
python -S prototype/forge_worker.py check vass \
  examples/mutex.model.json --certificate candidate.json

python -S prototype/forge_worker.py search nominal \
  examples/lru_fifo.model.json
python -S prototype/forge_worker.py search mixed \
  examples/owner_pool.model.json
```

The search CLI validates inputs, runs the producer, and checks any non-unknown result before printing it. Exit code zero means an accepted model result, including a valid negative witness; exit code two means unknown; exit code one means an input/checking error. An accepted result is still a Python-model result, not a Lean proof. The check-only path does not import producers.

The default experiment output directory is `reproduced-results`; it does not overwrite the retained evidence unless explicitly directed to do so. The recorded seed determines the generated corpus, but timings depend on the runtime and host. Article compilation uses pdfLaTeX with standard packages and no shell escape:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-state.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-state.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-state.tex
```

B Model and certificate schema notes

Counter model. The fields are `dimension`, `controls`, `initials`, `transitions`, and `bad`. An initial family contains `control`, `base`, and a list of `rays`. A transition contains `src`, `dst`, `consume`, and `produce`; a bad threshold contains `control` and `vector`. All entries have validated dimensions; negative integers and Python Booleans are not accepted as natural numbers.

Counter receipts. A positive object has `schema: 1`, `kind: "safe"`, and one `basis` list per control. A negative object has `kind: "unsafe"`, an `initial` family index, `parameters`, and original `transitions` indices. Debug `stats` are not evidence. The axes receipt’s mathematical payload is:

```
{"schema": 1, "kind": "safe", "basis": [[[1, 1]]]}
```

Equality models. A nominal input file is a pair of machine objects. Each supplies `register`, `constant`, and `control` counts; `tags`; `initial` control and anchored register values; and ordered rules. Selectors are `["input"]`, `["null"]`, `["reg", i]`, and `["const", j]`. Guards are Boolean

values or `eq`, `not`, `and`, and `or` syntax. Updates list one selector per register. Outputs carry a `bool` or `atom` tag and the appropriate expression. Fixed constants use labels $0, \dots, c - 1$.

Equality receipts. A positive object has `kind`: `"equivalent"` and a `states` list of `[leftControl, rightControl, jointRegisters]`. The checker requires distinct orbits, not necessarily canonical labels. A negative object has `kind`: `"different"` and a concrete `word` of `[tag, atom]` entries. It need not stop at the first mismatch to be valid; shortestness is a producer property tested separately, not part of receipt soundness.

Mixed models and receipts. A mixed model combines a single register-machine header with counter initial families and nondeterministic rules carrying `consume/produce`. Bad clauses carry input-independent register guards. A safe receipt has `states` entries `[control, registers]` and one counter basis per entry. An unsafe receipt has initial-family parameters and concrete `steps` triples `[ruleId, tag, atom]`. The receipt does not include a trusted compiled edge table.

Unknown. Search may return `kind`: `"unknown"` with a reason and statistics. The certificate checkers intentionally do not accept it as a proof. The JSON schema is a prototype interchange format; extra diagnostic fields are ignored, while the mathematical fields and model dimensions are checked. It is not a versioned public Forge protocol yet.

C Source audit and implementation status

Full baseline identifiers are:

```
Forge  58ea206bd7ad501b930add568151217bcc7f82a2
Leant  6bf05ad78c467989e68290f2d08bbcd40802d485
Djex   e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5
```

The inspected Forge README gives a Lean v4.34.0 baseline and Mathlib `1cf325a0cf67`; the full Mathlib identifier is

```
1cf325a0cf67aca2b04d76b5380ff6a9e410aefa
```

Leant's reported native Djex dependency is `e237e866`; that dependency identifier must not be silently replaced by the separate Djex repository-head identifier above.

The main Forge audit paths were:

```
README.md
article/sections/06-closure.tex
article/sections/15-conclusion.tex
article/sections/B-provenance.tex
lean/Forge/Design/Contracts.lean
lean/Forge/Design/Runtime.lean
lean/Forge/Closure/Covers.lean
```

The Leant current commit response included the complete indexing-composition diagnostic report; Djex's README supplied its checked synthesis scope and embedding entry points. Their archived materials are not reproduced in this package.

Artifact or assertion	Status	Evidence
Counter, nominal, mixed producers	Executed	Stored corpus and generator.

Artifact or assertion	Status	Evidence
Independent Python receipt replay	Executed	650 accepted objects; no producer imports in replay.
Differential finite-oracle checks	Executed	640 agreements, subject to described generator and evaluator sharing.
Exact fragment soundness and completeness	Mathematical arguments	Proofs in Sections 2–4; not formalized here.
Scalar/vector Lean specimen	Uncompiled	Source included, no <code>sorry</code> ; <code>NOT_RUN</code> , no claimed axiom output.
Lean certificate checker and source frontend	Proposed	Module boundaries, proof obligations, and acceptance gates only.
Leant/Djex semantic-summary integration	Proposed	Candidate-flow design; original indexing and universe queries remain untouched.
Performance versus Lean tactics	Not measured	No end-to-end tactic comparison or claimed solved-goal gain.

D Concluding assessment

The meaningful extension is a change in what Forge can finitely represent: upward regions of unbounded natural states, equality orbits of states containing arbitrary names, and the exact product of these representations under an explicitly restricted syntax. The proofs identify both the reason for finite certificates and the cases where those certificates cease to be complete.

The prototypes do more than demonstrate plausible examples. They discover positive receipts, produce executable failures, preserve explanations under subsumption and renaming, compare against exhaustive finite oracles on suitable fragments, and replay stored evidence without importing searches. They also leave the important boundary visible: a model theorem becomes a Lean source theorem only through a verified checker and a proved translation. Building that narrow path is the recommended next contribution to Forge.

References

- [1] Vladimir Reshetnikov, *Forge*, inspected revision 58ea206bd7ad, 15 September 2026. README and merged article/provenance ledger. Pinned repository source.
- [2] Forge, *Logical certificate contracts*, same revision, `lean/Forge/Design/Contracts.lean`. Pinned repository source.
- [3] Forge, *Runtime and scheduler design contracts*, same revision, `lean/Forge/Design/Runtime.lean`. Pinned repository source.
- [4] Vladimir Reshetnikov, *Leant*, revision 6bf05ad78c46. *Indexing composition after native kind adoption*, 14 September 2026. Pinned repository source.
- [5] Vladimir Reshetnikov, *Djex*, README at revision e8778f4ebd63. Pinned repository source.
- [6] Parosh Aziz Abdulla, Karlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay, *Algorithmic Analysis of Programs with Well Quasi-ordered Domains*, Information and Computation 160(1–2), 109–127, 2000. DOI: 10.1006/inco.1999.2843.

- [7] Alain Finkel and Philippe Schnoebelen, *Well-structured transition systems everywhere!*, Theoretical Computer Science 256(1–2), 63–92, 2001. DOI: 10.1016/S0304-3975(00)00102-X.
- [8] Mikołaj Bojańczyk, Bartek Klin, and Sławomir Lasota, *Automata theory in nominal sets*, Logical Methods in Computer Science 10(3:4), 2014. <https://lmcs.episciences.org/1157>. DOI: 10.2168/LMCS-10(3:4)2014.
- [9] Mathlib documentation, *Mathlib.Order.WellQuasiOrder*, accessed 15 September 2026. Rolling documentation, not a pinned compilation result. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Order/WellQuasiOrder.html.
- [10] LIS development team, *MinCoverPetri*, official application documentation, accessed 15 September 2026. <https://dev.pages.lis-lab.fr/mincoverpetri/>. The documented algorithm is Pierre-Alain Reynier and Frédéric Servais, *Minimal Coverability Set for Petri Nets: Karp and Miller Algorithm with Pruning*, Fundamenta Informaticae 122(1–2), 1–30, 2013. DOI: 10.3233/FI-2013-781.
- [11] LearnLib project, *RaLib: Library for active learning algorithms for register automata*, official repository, accessed 15 September 2026. <https://github.com/LearnLib/ralib>.
- [12] Mikołaj Bojańczyk, Joanna Fijalkow, Bartek Klin, and Joshua Moerman, *Orbit-Finite-Dimensional Vector Spaces and Weighted Register Automata*, TheoretiCS 3, article 13, 2024. <https://theoretics.episciences.org/13547>. DOI: 10.46298/theoretics.24.13.